COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	15/10/2026	**DATE SUBMITTED:**	19/10/2026
NAME:	DYLAN PAOLO M. MAGTAAN Serrano, Gabriel Immanuel R.	**INSTRUCTOR:**	ENGR. JIMLORD QUEJADO

ACTIVITY NO. 13.1 **Parallel Algorithms and Multithreading**

1. PROCEDURE OUTPUT

ILO A & ILO B:

Part 1: Simple One-Threaded Example

Since all threads must be given a function to complete at their creation, we first must declare a function for it to be given. We'll name this function print and will design it to take int and string arguments when called.
When executed, this code will simply report the data values passed in.

```
void print(int n, const std::string &str) {
    std::cout << "Printing integer: " << n <<
    std::endl; std::cout << "Printing string: " <<
    str << std::endl;
}
```

In the next section, we'll initialize a thread and have it execute the above function. To do this, we'll have the main function, the default executor present in all C++ applications, initialize the thread for the print function.

After that, we use another handy multithreading command, `join()`, pausing the main function's thread until the specified thread, in this case t1, has finished its task. Without `join()` here, the main thread would finish its task before t1 would complete print, resulting in an error.

```
int main() {
    std::thread t1(print, 10,
    "T.I.P."); t1.join();
return 0;
}
```

Show your output and analysis in section 6 as its own table. Part 2:

Screenshot	SingleThread > main.cpp > ...
	1 #include <iostream>
	2 #include <thread>
	3
	4 void print(int n, const std::string &str) {
	5 std::cout << "Printing integer: " << n << std::endl;
	6 std::cout << "Printing string: " << str << std::endl;
	7 }
	8
	9 int main() {
	10 std::thread t1(print, 10, "T.I.P.");
	11 t1.join();
	12 return 0;
	13 }
	Printing integer: 10
	Printing string: T.I.P.

Analysis	The program demonstrates the use of a single thread by creating a separate thread to execute the print function. The function receives the integer value of 10 and a string "TIP", then displays both values. The join function ensures that the main program waits for the thread to finish executing before proceeding to terminate the program.

Table 13-1. Simple One-Threaded Example

Multi-Threaded Example

While the outcome of the single thread example above could easily be replicated without using multithreaded
code, we can truly see concurrency's benefits when we attempt to complete print multiple times with different sets of data. Without multithreading, this would be done by simply having the main thread repeat print one at a time until completion.

To do this with concurrency in mind, we instead use a for loop to initialize

multiple threads, pass them the print function and arguments, which they then complete concurrently. This multithreading option would be faster one using only the main thread as more of the total CPU is being used.

Runtime difference between multithreading and non-multithreading solutions increasing as more print executions are needed.

Let's see what a many-thread version of the above code would look like:

```
void print(int n, const std::string &str) {
  std::string msg = std::to_string(n) + " :
  " + str; std::cout << msg << std::endl;
}

int main() {
  std::vector<std::stri
  ng> s = {
       "T.I.P.",
       "Competen
       t",
       "Computer
       ",
       "Engineer
       s"
  };
  std::vector<std::thread> threads;

  for (int i = 0; i < s.size(); i++) {
    threads.push_back(std::thread(print, i, s[i]));
  }

  for (auto &th :
    threads) {
    th.join();
  }
  return 0;
}
```

Show your output and analysis in section 6 as its own table.

Screenshot

MultiThread > main.cpp > ...

```cpp
#include <iostream>
#include <vector>
#include <thread>

void print(int n, const std::string &str) {
    std::string msg = std::to_string(n) + " : " + str;
    std::cout << msg << std::endl;
}

int main(){
    std::vector<std::string> s ={
        "T.I.P.",
        "Competent",
        "Computer",
        "Engineers"
    };
    std::vector<std::thread> threads;

    for(int i = 0; i < s.size(); i++){
        threads.push_back(std::thread(print, i, s[i]));
    }
    for (auto &th : threads){
        th.join();
    }
    return 0;

}

```

Serrano	Magtaan
3 : Engineers 2 : Computer 0 : T.I.P. 1 : Competent	0 : T.I.P.1 : Competent2 : Computer3 : Engineers

Analysis

The program demonstrates multithreading by creating separate threads to print each string in the vector. The print function combines an integer index and a string into a message, while the for loop creates four threads to process the string simultaneously. As you can see on the results since the threads execute independently, the output may vary depending on which thread finishes first, it also depends on the computer hardware. Then the join function ensures that the main program waits for all the threads to finish before terminating the program.

Table 13-2. Multithreaded Example.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Part A: Demonstrate an understanding of parallelism, concurrency, and multithreading in C++ by answering the given questions. Use of supplementary materials to support answers must be cited as reference.

Questions:

1. Write a definition of multithreading and its advantages/disadvantages

Magtaan

- Multithreading allows a single process to execute multiple threads concurrently. This enables faster execution and the ability to run multiple tasks at the same time. However, this could cause possible race conditions when threads access shared data.

Serrano

- Multithreading allows a program to execute multiple threads concurrently. Its advantages include faster execution and efficient handling of multiple tasks, while its disadvantages include synchronization issues, improper thread coordination, and race conditions that occur when multiple threads access shared data simultaneously.

2. Rationalize the use of multithreading by providing at least 3 use-cases.

Magtaan

- Multithreading is useful in situations where a program needs to perform multiple tasks concurrently.
    1. Web servers
        - Allows multiple users to send requests and receive responses at the same time.
    2. Video games
        - Allows processing graphics, handling user input, and game logic, to run concurrently for smoother gameplay.
    3. File processing
        - Allows a program to read, process, or download multiple files simultaneously.

Serrano

1. Messaging Applications
    - This allows users to send and receive messages while notifications and file transfers are handled simultaneously.
2. Video Games

- This allows graphics, sounds, and player input to be processed concurrently.

3. Multimedia Applications
   - This allows audio playback, video rendering, and user controls to work at the same time.

3. Differentiate between parallelism and concurrency.

Magtaan
- Concurrency is when a program manages multiple tasks using threads that take turns executing on a single CPU core. Parallelism is when multiple tasks execute simultaneously using different CPU cores. Concurrency focuses on managing multiple tasks efficiently, whereas parallelism focuses on executing tasks at the same time to improve performance.

Serrano
- Concurrency allows multiple tasks to make progress during overlapping periods by taking turns executing, while parallelism involves multiple tasks executing at the same time using multiple processors or CPU cores. In terms of speed, parallelism can complete tasks faster by performing them at the same time, while concurrency improves efficiency by managing multiple tasks within the same time.

Part B: Create C++ Code and show a solution that satisfies the given requirements below.

- Create a global variable of type integer.
- Create an add function that will take an integer parameter and add that value to the global variable.
- Use multi-threading techniques to create 3 threads; individually pass the add function to the threads.
- Display the value of global variable at different combinations of using the join() per thread. Such that:
  - Display
  - T1.join()
  - Display
  - T2.join()
  - Display
  - T3.join()
- Provide an analysis based on the outputs of the multi-threading exercise.

Code (Magtaan)

```cpp
#include <iostream>
#include <thread>

int globalVar = 0;

// Multithreading
void add(int n) {
    globalVar += n;
}

int main() {
    std::thread T1(add, 10);
    std::thread T2(add, 20);
    std::thread T3(add, 30);

    std::cout << "Initial value: " << globalVar << std::endl;

    T1.join();
    std::cout << "After T1.join(): " << globalVar << std::endl;

    T2.join();
    std::cout << "After T2.join(): " << globalVar << std::endl;

    T3.join();
    std::cout << "After T3.join(): " << globalVar << std::endl;

    return 0;
}
```

Theoretical Analysis (Magtaan)

This program demonstrates multithreading by creating three threads, each adding a different value to a shared global variable. The join() function makes the main thread wait for each specified thread to finish before printing the updated value. However, since all three threads modify globalVar concurrently without synchronization, a data race can occur, making the final result unpredictable.

Experimental Analysis (Magtaan)

```
Initial value: 0
After T1.join(): 60
After T2.join(): 60
After T3.join(): 60
```

The initial value of globalVar was 0, but after T1.join(), the value was already 60 because all three threads had finished adding 10, 20, and 30. The value remained 60 after T2.join() and T3.join() because all additions were already completed.

Code (Serrano)

```cpp
#include <iostream>
#include <thread>

int globalVar = 0;

// Multithreading
void add(int n) {
    globalVar += n;
}

int main() {
    std::thread T1(add, 10);
    std::thread T2(add, 20);
    std::thread T3(add, 30);

    std::cout << "Initial value: " << globalVar << std::endl;

    T1.join();
    std::cout << "After T1.join(): " << globalVar << std::endl;

    T2.join();
    std::cout << "After T2.join(): " << globalVar << std::endl;

    T3.join();
    std::cout << "After T3.join(): " << globalVar << std::endl;

    return 0;
}
```

```
Initial value: 0
After T1.join(): 60
After T2.join(): 60
After T3.join(): 60
PS C:\Users\Gabriel\Desk
```

Theoretical Analysis
The program creates three threads that execute the add() function to add 10, 20, and 30 to the global variable globalVar. The time complexity is O(1) because each thread performs a single addition regardless of input size. The join() function ensures that each thread finishes execution before the program continues to the next statement.
Experimental Analysis
The initial value of globalVar may be 0 or already 60, depending on whether the threads have finished adding their values before the first print. After all three join() calls, the value should be 60 because all threads have finished adding.

Part C: Use multi-threading with one of the algorithms previously developed in the course; provide an analysis of the result.

Picture

```
template <typename T>
void merge(T arr[], int left, int middle, int right) {
    int leftSize = middle - left + 1;
    int rightSize = right - middle;

    T* leftArray = new T[leftSize];
    T* rightArray = new T[rightSize];

    for (int i = 0; i < leftSize; i++) {
        leftArray[i] = arr[left + i];
    }

    for (int i = 0; i < rightSize; i++) {
        rightArray[i] = arr[middle + 1 + i];
    }

    int i = 0;
    int j = 0;
    int k = left;

    while (i < leftSize && j < rightSize) {
        if (leftArray[i] <= rightArray[j]) {
            arr[k] = leftArray[i];
            i++;
        }
        else {
            arr[k] = rightArray[j];
            j++;
        }
        k++;
    }
```

```
    while (i < leftSize && j < rightSize) {
        if (leftArray[i] <= rightArray[j]) {
            arr[k] = leftArray[i];
            i++;
        }
        else {
            arr[k] = rightArray[j];
            j++;
        }
        k++;
    }

    while (i < leftSize) {
        arr[k] = leftArray[i];
        i++;
        k++;
    }

    while (j < rightSize) {
        arr[k] = rightArray[j];
        j++;
        k++;
    }

    delete[] leftArray;
    delete[] rightArray;
}
```

```
// Merge Sort with Multithreading
template <typename T>
void mergeSort(T arr[], int left, int right, int depth = 0) {
    if (left < right) {
        int middle = left + (right - left) / 2;

        if (depth < 2) {
            std::thread t1([&]() {
                mergeSort(arr, left, middle, depth + 1);
            });

            mergeSort(arr, middle + 1, right, depth + 1);

            t1.join();
        }
        else {
            mergeSort(arr, left, middle, depth + 1);
            mergeSort(arr, middle + 1, right, depth + 1);
        }

        merge(arr, left, middle, right);
    }
}
```

Analysis (Magtaan)
Multithreading was used in the merge sort algorithm to sort the left and right halves of the array concurrently. The std::thread creates a separate thread to process the left half while the main thread sorts the right half. t1.join() ensures that the left half finishes sorting before both halves are merged. The thread creation is limited to the first two recursion levels to reduce overhead.
Analysis (Serrano)
The algorithm uses Merge Sort to divide the array into smaller parts, sort them, and merge them back in order. Its time complexity is O(n log n) in the best, average, and worst cases because the array is repeatedly divided and merged. Multithreading allows parts of the array to be sorted at the same time,the std::thread creates a separate thread to sort the left half while the main thread sorts the right half, and t1.join() ensures that both halves finish sorting before they are merged.

3. Conclusion

Magtaan

Summary of lessons learned
I learned how to create and manage threads in C++ using std::thread and join(). I also learned that multithreading allows multiple tasks to execute concurrently, although the output order may vary.
Analysis of the procedure
I observed how a single thread executes a function and how multiple threads can print different strings concurrently. Using join() ensures that all threads finish before the program ends.
Analysis of the supplementary
The supplementary helped me apply multithreading to Merge Sort by sorting separate portions of an array concurrently. I limited thread creation to the first two recursion levels to reduce overhead and used sequential calls for deeper levels.
How well did you think you did in this activity?
I think I did well because I understood the basic concepts of multithreading and applied them to a sorting algorithm in the supplementary.
What are your areas for improvement?
Doing the supplementary I tried using multithreading on merge sort without limiting the depth. Later, I realized that the standard was to limit it to higher levels to ensure minimal overhead. Therefore, I think

I need to improve my understanding of performance measurement to determine when multithreading is more efficient than sequential execution.

Serrano

Summary of lessons learned

I learned how multithreading and parallelism work and how they can help a program perform multiple tasks at the same time. I also learned how to use std::thread and join() and how data races can affect the results of a program. Lastly, I learned that the execution time may vary depending on the computer's hardware and how the threads are executed.

Analysis of the procedure

The procedure helped me understand how to create and manage threads. I learned how join() makes sure a thread finishes its task before the program continues and how multithreading can be applied to different algorithms and how different output is expected even though you arrange them properly because of the hardware.

Analysis of the supplementary

For the supplementary activity, I learned how to utilize multithreading by creating three threads that add values to a global variable and for the other part where merge sort was utilized with multithreading. It shows me that multithreading can help sort parts of an array at the same time, but it might cause some errors where the data access the same variable at the same time.

How well did you think you did in this activity?

I think I did well in this activity because I was able to understand the basic concepts of multithreading and apply them to the codes. I was also able to analyze the time complexity of multithreading merge sort.

What are your areas for improvement?

I need to improve my understanding of how threads work and how to prevent data from accessing the same memory location at the same time. I also need to practice more on applying multithreading and parallel algorithms in our previous HOA like linked list, merge sort and many more.

4. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT and Grammarly**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

COMPUTER ENGINEERING

QUEZON CITY

Purpose of Use: **Grammar correction and structure, and code debugging.**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **AI was only used for grammar correction and structure, and code debugging. All ideas and analysis remain original from us.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.